جزوهٔ درسی · جلسهٔ ۲ از ۵

ابزارهای عامل و تعامل‌پذیری

پروتکل‌هایی که عامل (agent) را از گاراژ بیرون می‌آورند

مطالعهٔ پیشنهادی: ۵۰ دقیقه **پیش‌نیاز:** جزوهٔ جلسهٔ اول

فهرست مطالب

1. چرا پروتکل؟ از ماشین سفارشی تا استاندارد صنعتی
2. MCP از نگاه کدنویس شهودی
3. مسئلهٔ N×M و حامل‌ها
4. دیباگ و جعبه‌ابزار مصرف امن
5. تکامل معماری‌های عامل‌محور
6. A2A: تعامل‌پذیری عامل به عامل
7. A2UI: رابط کاربری به‌عنوان خروجی عامل
8. تجارت عامل‌محور: UCP و AP2
9. جمع‌بندی
10. خودآزمایی

۱. چرا پروتکل؟ از ماشین سفارشی تا استاندارد صنعتی

> نسل بعدی نرم‌افزار نوشته نمی‌شود؛ توسط عامل‌های تعامل‌پذیر کارگردانی می‌شود.

در جلسهٔ اول دیدیم که محصول اصلی کار توسعه‌دهنده دیگر کدِ خام نیست، بلکه سامانه‌ای است که کد را تولید می‌کند. اگر مهندسی عامل‌محور (Agentic engineering) همان کارخانه‌ای است که شما اداره‌اش می‌کنید، MCP، A2A، A2UI، AP2 و UCP همان **استانداردهای صنعتی** آن‌اند: اندازهٔ یکسان پیچ‌ومهره، قالب‌های داده و کانال‌های ارتباطی که اجازه می‌دهند ماشین‌آلات شما با بقیهٔ جهان ایمن تعامل کند.

هزینهٔ نداشتن استاندارد

بدون پروتکل‌های باز، هر عاملی که می‌سازید یک «ماشین سفارشی» منزوی در گاراژ است. ناچارید برای هر ابزار (tool) و هر اتصال API پوشش‌های (coverage) شکننده و اختصاصی بنویسید و نگه‌داری کنید. هر API می‌شود «استانداردی برای یک نفر» و این یعنی بدهی فنی خالص — کارهایی کم‌بازده که شما را در نقش رهبر ارکستر (Conductor) حبس می‌کنند.

پنج استاندارد و نقششان

استاندارد	استعاره	نقش
MCP پروتکل زمینهٔ مدل (MCP)	پریز USB-C درون مهار (harness)	اتصال فوری مدل‌ها به پایگاه‌های داده، فایل‌سیستم‌ها و APIهای وب.
مهارت‌ها (skill)	دفترچهٔ راهنما	دستورها و اسکریپت‌های ساده‌ای که در محیط سندباکس (sandbox) اجرا می‌شوند. موضوع جلسهٔ سوم.
A2A عامل به عامل	بی‌سیم کارخانه	عامل‌های متخصص با آن مذاکره و هم‌فکری می‌کنند و کار را به هم می‌سپارند.
A2UI عامل به رابط کاربری	ویترین مولد	تبدیل خروجی خام و پیچیده به مؤلفه‌های بصری امن و تعاملی برای اپراتور انسانی.
AP2 و UCP	شبکهٔ تأمین و تراکنش جهانی	مذاکره و اجرای ایمن تراکنش‌های تجاری خودگردان (autonomous).

تمرکز این جلسه: مصرف، نه ساخت

در مقالهٔ پیشین این مجموعه، معماری سازمانی MCP — توپولوژی میزبان-کلاینت-سرور، ساخت سرور سفارشی و حاکمیت امنیتی — تشریح شده بود. اما برای کدنویس شهودی، اولویت **مصرف است بر ساخت**. شما نمی‌خواهید ساعت‌ها پیکربندی سرور بنویسید؛ می‌خواهید به فهرست‌های عمومی و خصوصی موجود وصل شوید و فوری به عامل ابرقدرت «وصل‌کن و کار کن» بدهید.

۲. MCP از نگاه کدنویس شهودی

روش سنتی سازمانی برای رساندن ابزار به مدل، سیم‌کشی دستی و اختصاصی است: هر اتصال تازه یعنی پوشش سفارشی REST، کلید API، سیاست (policy) تازه‌سازی توکن (token) و تجزیه‌گر سفارشی JSON. MCP همهٔ این اصطکاک را با یک سوکت استاندارد جایگزین می‌کند. راه‌اندازی در سه گام ساده انجام می‌شود.

گام ۱ — کشف

کدنویس شهودی اتصال‌دهندهٔ سفارشی نمی‌نویسد؛ سرورهای از پیش ساخته‌شده را از سه منبع اصلی پیدا می‌کند:

- **فهرست‌های عمومی MCP:** صدها سرور آماده میزبانی می‌کنند. برای نمونه‌سازی سریع محلی عالی‌اند، اما **بازبینی‌نشده‌اند** و استفاده‌شان با مسئولیت خودتان است.
- **سرورهای راه دور شخص ثالث معتبر:** سکوهای بازبینی‌شده که نقاط پایانی میزبانی‌شده عرضه می‌کنند؛ مثلاً سرورهای رسمی منتشرشده برای سرویس‌های مدیریت‌شده که به عامل اجازه می‌دهند امن با آن سرویس‌ها تعامل کند.
- **فهرست‌های داخلی:** درون سازمان، ابزارهای داخلی به‌شکل سرور MCP عرضه و در فهرستی امن ثبت می‌شوند. پیاده‌سازی فنی این فهرست معمولاً از راه یک دروازهٔ API، فهرست ثبت (Registry) عامل، یا درگاه ریزسرویس خصوصی مدیریت می‌شود.

> **نکتهٔ کاربردی امنیتی**
>
> هنگام تصمیم‌گیری، همیشه امنیت را اولویت نخست بگذارید و تا جای ممکن سراغ دستورالعمل‌های رسمی بروید. اگر از سرورهای عمومی جامعه استفاده می‌کنید، **اعتبارنامه** (credential) **ندهید** و از سرویس‌های محافظ برای پیشگیری از مسائل امنیتی استفاده کنید.

گام ۲ — پیکربندی

پیش از اتصال، پارامترهای استاندارد دامنه و مجوزهای سرور را مشخص کنید:

- بررسی پیش‌نیازها.
- تعیین دامنه و معیارهای دسترسی.
- گنجاندن مشخصات (spec) در عامل کدنویسی.
- احراز هویت (identity): تنظیم فایل‌های محیطی برای مدیریت اعتبارنامه‌ها (مانند توکن‌های دسترسی شخصی یا رازهای OAuth) و تعریف مجوزهای خواندن و نوشتن برای محافظت از فایل‌سیستم.

گام ۳ — اتصال

پس از پیکربندی، کلاینت اتصال را با سرور برقرار می‌کند. برای راستی‌آزمایی، کلاینتِ میزبان یک درخواست دست‌دادن ساده اجرا می‌کند تا ابزارهای موجود را فهرست کند و اسکیمای خروجی را اعتبارسنجی کند.

۳. مسئلهٔ N×M و حامل‌ها

برای فهم اینکه چرا MCP برای نمونه‌سازی سریع حیاتی است، به ریاضیات یکپارچه‌سازی نگاه کنید. اگر با N مدل زبانی مختلف آزمایش می‌کنید و می‌خواهید آن‌ها را به M ابزار بیرونی وصل کنید، توسعهٔ سنتی برای **هر تقاطع مدل و ابزار** کد یکپارچه‌سازی سفارشی می‌خواهد.

حساب ساده

با ۵ مدل و ۱۰ ابزار باید ۵۰ نقطهٔ یکپارچه‌سازی اختصاصی را نگه دارید. اگر API یک ابزار عوض شود، چند حلقهٔ تجزیه (decomposition) هم‌زمان می‌شکنند.

با پروتکل: پیچیدگی از O(N × M) به O(N + M) کاهش می‌یابد.

حامل‌های استاندارد

با استانداردشدن تعریف ابزارها، حامل‌ها (Transport) را بدون نوشتن کد یکپارچه‌سازی مستقیماً به مهار عامل وصل می‌کنید:

حامل	سازوکار	مزایا و ملاحظات
stdio ورودی/خروجی استاندارد	کلاینت میزبان، سرور MCP را به‌عنوان زیرفرایند محلی در پس‌زمینه اجرا می‌کند و پیام‌های JSON-RPC 2.0 را از راه stdin و stdout رد و بدل می‌کند.	بیشتر برای کار محلی و نمونه‌سازی. عامل بدون پیکربندی پیچیدهٔ شبکه کار می‌کند و ابزار را مثل یک فرایند محلی می‌بیند.
SSE روی HTTP رویدادهای ارسالی از سرور	کلاینت میزبان (محلی یا یک برنامهٔ عامل‌محور مستقر) به نقطهٔ پایانی راه دور وصل می‌شود و داده به‌صورت زمان‌واقعی به عامل جاری می‌شود.	وابستگی (dependency) کمتر، همیشه به‌روز، ردپای کوچک‌تر و چرخهٔ عمر ساده‌تر؛ اما بار بیشتری روی سرور ابری می‌گذارد.

۴. دیباگ و جعبه‌ابزار مصرف امن

وقتی عامل توهم می‌زند، لوله‌ها را ببینید

وقتی عامل پارامتر توهم (hallucination) می‌زند، ابزار اشتباه را صدا می‌کند یا نمی‌تواند بار داده را تجزیه کند، **وقتتان را صرف دست‌کاری کورکورانهٔ سیستم پرامپت‌ها نکنید.** مستقیم لوله‌های انتقال را دیباگ (debug) کنید:

- **MCP Inspector:** ابزار بومی توسعه‌دهنده که یک پنل وب محلی اجرا می‌کند. با آن می‌توانید هر سرور محلی یا راه دور را دستی پرس‌وجو کنید، اسکیماهای ابزار فعال را ببینید، ورودی‌ها را دستی بیازمایید و بسته‌های خام JSON-RPC 2.0 را بازرسی کنید — بی‌آنکه گردش‌کار (workflow) اصلی عامل را راه بیندازید.
- **Chrome DevTools:** مناسب محیط‌های توسعهٔ وب‌محور و دیباگ اتصال‌های SSE؛ امکان ردیابی جریان‌های ورودی و بررسی تأخیر (latency) سرور.

بایدها

- **سرورهای عمومی را پیش از اتصال ممیزی کنید.** پیش از وصل‌کردن سرور متن‌باز عمومی به عاملی که به فایل‌سیستم یا اعتبارنامه‌های شما دسترسی دارد، کدش را بخوانید.
- **برای ابزارها از بازیابی (retrieval) پویا (dynamic) استفاده کنید.** کانتکست ویندو (context window) را تمیز نگه دارید: ابزار را فقط هنگام نیاز از فهرست بارگذاری کنید و پس از پایان کار از زمینه (context) بیرون بیندازید تا توجه رقیق نشود.
- **از دروازه‌ها (gate) و فهرست‌های داخلی بهره بگیرید.** این یعنی مصرف اسکیماهای تأییدشده و حکمرانی‌شده، به‌جای اختراع دوبارهٔ چرخ یا اجرای کد بازبینی‌نشده.
- **از بازرس استفاده کنید.** وقتی عامل فراخوان ابزاری را با آرگومان‌های ساختگی می‌زند، به‌جای دست‌کاری کورکورانهٔ پرامپت (prompt) سامانه، دادهٔ خام انتقال را ببینید.
- **انسان را در حلقه (loop) نگه دارید.** ورودی ابزار را پیش از فراخوانی سرور به کاربر نشان دهید تا از خروج تصادفی یا مخربِ داده جلوگیری شود.
- **برای ممیزی ثبت کنید.** استفاده از ابزارها را گزارش‌برداری کنید.

نبایدها

- **اگر می‌شود مصرف کرد، نسازید.** در برابر وسوسهٔ نوشتن پوشش سفارشی REST مقاومت کنید؛ اول دنبال سرور موجود بگردید.

- **سرور عمومی و تأییدنشده را در تولید به کار نبرید.** برای نمونهٔ آخر هفته عالی‌اند، اما ریسک امنیتی و پایداری دارند. منطق اصلی کسب‌وکار را به نقطهٔ پایانی تأییدنشده گره نزنید.
- **اعتبارنامه را سخت‌کد نکنید.** هرگز کلید یا توکن را مستقیم در پرامپت عامل یا اسکریپت محلی نچسبانید؛ از متغیرهای محیطی استفاده کنید.
- **به محیط تولید وصل نشوید.** سرور را با پروژهٔ توسعه به کار بگیرید، نه تولید. مدل‌ها در طراحی و تست (test) عالی‌اند؛ از آن‌ها در محیطی امن و با دادهٔ غیرتولیدی یا مبهم‌سازی‌شده استفاده کنید.
- **برای به‌روزرسانی به کارش نبرید.** اگر ناچارید به دادهٔ واقعی وصل شوید، سرور را در حالت فقط‌خواندنی بگذارید.
- **دسترسی گسترده به همهٔ پروژه‌ها ندهید.** دامنهٔ سرور را به یک پروژهٔ مشخص و منابع همان پروژه محدود کنید.

۵. تکامل معماری‌های عامل‌محور

برای فهم جایگاه کدنویسی شهودی (Vibe coding) در تکامل معماری‌های عامل‌محور، می‌توان الگویی تکرارشونده در فناوری را دید: گذار از پیکربندی دستی و سطح‌پایین، به هماهنگی اعلانی و مبتنی بر قصد (intent) — جایی که کاربر نمی‌گوید **چگونه** چیزی را بساز، بلکه می‌گوید **چه** می‌خواهد. همین روند در زیرساخت فناوری اطلاعات (گذار از پیکربندی دستی به «زیرساخت به‌مثابه کد») و در یادگیری ماشین (وعدهٔ AutoML برای خودکارکردن کارِ پرزحمت ساخت مدل) رخ داده است.

سقف معماری یکپارچه

امروز می‌توان کل یک برنامه را با کدنویسی شهودی ساخت. اما مسیر (trajectory) معماری‌های عامل‌محوری که این برنامه‌ها را می‌سازند، دارد یکی از مهم‌ترین گذارهای تاریخ نرم‌افزار را بازتاب می‌دهد: گذار از معماری یکپارچه به ریزسرویس‌ها.

همان‌طور که برنامه‌های وب اولیه کدبیس‌هایی یگانه و غول‌آسا بودند که در آن‌ها همهٔ کارکردها به هم چسبیده بود، بسیاری از پروژه‌های اولیهٔ کدنویسی شهودی هم بر «عامل یکپارچهٔ چاقوی‌همه‌کاره» تکیه می‌کنند: یک پرامپت (prompt) بسیار پیچیده که در آن یک عامل با ده‌ها ابزار متصل، همه‌چیز را از پرس‌وجوی پایگاه داده تا رندر رابط و تست بر عهده می‌گیرد. این معماری اجازه می‌دهد در یک آخر هفته نمونه‌ای سرِهم شود، اما سرانجام به همان «سقف یکپارچگی» می‌خورد:

محدودیت	چه اتفاقی می‌افتد
اصطکاک مقیاس	نمی‌توانید «منطق پایگاه داده» را بهینه کنید بدون آنکه «منطق رابط کاربری» را که در همان پرامپت است گیج کنید. افزون بر آن، هرچه ابزارهای در دسترس یک عامل بیشتر باشد، تصمیم‌گیری‌اش بدتر می‌شود: فضای جست‌وجوی کنش بعدی بیش‌ازحد بزرگ می‌شود و به پارامترهای ساختگی یا فراخوانی ابزار اشتباه می‌انجامد.
اضافه‌بار زمینه‌ای	چپاندن دستورهای کلان سامانه، ده‌ها اسکیمای پیچیدهٔ ابزار و تاریخچهٔ جاری گفت‌وگو در یک پرامپت، به‌سرعت حافظهٔ کاری مدل را پر می‌کند.
نقطهٔ شکست یکتا	یک اشکال در یک ابزار یا یک دستور (instruction) می‌تواند کل عامل را به توهم یا سقوط بکشاند. دادهٔ نامربوط یا خراب که در زمینه ذخیره شده، منتقل می‌شود و استدلال‌های بعدی را هم می‌شکند.

درسی که پیش‌تر آموخته شده بود

وقتی مدل‌های اولیهٔ AutoML ارزش کسب‌وکاری‌شان را ثابت کردند، تیم‌ها به‌ندرت آن جعبه‌سیاه‌های خودتولید را در قلب کسب‌وکار رها کردند. برای رسیدن به مقیاس تولید، ناچار شدند جعبه را به مراحل قابل‌مشاهده و قابل‌نگه‌داری بشکنند — نسخه‌بندی داده، انبار ویژگی، تشخیص رانش. با تخصصی‌کردن اجزا، قانون بنیادی طراحی سامانه را به کار بستند: **تخصص، سازوکار مقیاس است.**

تخصص‌گرایی داخلی

تخصص‌گرایی داخلی (Internal specialization) یعنی افراز منطقی یک عامل یکپارچه (Single-agent monolith) به زیرعامل‌های (subagent) متمایز و هدفمند. هر عامل با یک سیستم پرامپت بسیار متمرکز و زیرمجموعه‌ای مرتبط از ابزارها اداره می‌شود. این ساختار همچنان یکپارچه است، چون این زیرعامل‌های داخلی از مرز شبکه عبور نمی‌کنند؛ همان رانتایم (runtime) و همان حافظهٔ زیرین را به اشتراک می‌گذارند. نتیجه: ماژولارشدن تعریف‌های پیچیده، در کنار حفظ تأخیر پایین و مدیریت سادهٔ وضعیتِ یک برنامهٔ تک‌فرایندی. سه بهبود فنی مشخص:

- **کوچک‌شدن فضای جست‌وجو:** با محدودکردن ابزارها و مهارت‌های هر زیرعامل (مثلاً دادن فقط ابزارهای پرس‌وجو به عامل پایگاه داده)، خطاهای فراخوان ابزار (tool call) و توهم به‌شدت کم می‌شود.
- **مهار رقیق‌شدن توجه** (Attention dilution)**:** تخصص به مدل زیرین اجازه می‌دهد روی یک دامنهٔ واحد تمرکز کند و استدلال تیزتری داشته باشد.

- **بهینه‌شدن بار زمینه‌ای:** چون هماهنگ‌کننده (Orchestrator) به‌جای پردازش کل درخت منطق فقط کارها را مسیریابی (routing) می‌کند، زیرعامل‌ها نسبت سیگنال به نویز بالایی در کانتکست ویندوشان حفظ می‌کنند.

معماری توزیع‌شدهٔ چندعاملی و پرسش «بسازیم یا بخریم»

با بلوغ معماری‌های عامل‌محور، جابه‌جایی چشمگیری به‌سوی معماری‌های توزیع‌شده در جریان است. رهبران صنعت از عرضهٔ صرف API فراتر رفته‌اند و عامل‌های هوش مصنوعی دامنه‌ای منتشر می‌کنند که در اکوسیستم اختصاصی خودشان با تسلط کامل کار می‌کنند.

از نظر فنی ممکن است سامانه‌ای چندعاملی با زیرعامل‌های سفارشی برای سکوهای شخص ثالث بسازید، اما این کار **هزینهٔ نگه‌داری** سنگینی به همراه دارد: با انتخاب توسعهٔ اختصاصی متخصص‌های دامنه، مسئولیت کامل به‌روزرسانی منطق پرامپت و تعریف ابزارها در برابر تغییرات محصول و اسکیمای API بالادست را خودتان بر عهده می‌گیرید. راهبرد معماری بهتر، بهره‌گیری از **عامل‌های متخصص رسمی** است که تیم‌های دارای تخصص عمیق دامنه نگه‌شان می‌دارند.

> **اما گلوگاه تازه: پراکندگی**
>
> هر یک از این متخصص‌ها را ممکن است تیمی متفاوت با فناوری متفاوت ساخته باشد: یکی با یک چارچوب، دیگری با چارچوبی دیگر، سومی با چیزی کاملاً سفارشی. بیرون از مرزهای شبکهٔ شما هستند، زبان‌های متفاوتی حرف می‌زنند، ساختار بار متفاوتی انتظار دارند، وضعیت گفت‌وگو را متفاوت پردازش می‌کنند و به لایه‌های انتقال گوناگون تکیه دارند. اگر برای هر متخصص ناچار باشید کد یکپارچه‌سازی سفارشی و حلقهٔ تصحیح خطای اختصاصی بنویسید، مفهوم «تیم مجازی» فوری به یک پروژهٔ یکپارچه‌سازی بدل می‌شود.

۶. A2A: تعامل‌پذیری عامل به عامل

این آشفتگیِ پراکندگی، دقیقاً همان چیزی است که پروتکل A2A برای استانداردکردنش ساخته شده. این پروتکل که ابتدا در گوگل توسعه یافت و سپس به بنیاد لینوکس اهدا شد، لایه‌ای جهانی از ارتباط برای سامانه‌های عامل‌محور معرفی می‌کند. A2A زبان مشترک اکوسیستم هوش مصنوعی است: ظرافت‌های حامل شبکه، چارچوب‌های زیرین، زبان‌های برنامه‌نویسی و تفاوت‌های ساختار بار را انتزاع می‌کند. **همان‌طور که HTTP وب را استاندارد کرد، A2A نیروی کار مجازی را استاندارد می‌کند.**

چرا متخصص را نمی‌توان مثل ابزار صدا زد؟

> «آیا فراخوان‌کننده به یک **نتیجه** نیاز دارد، یا به **شرکت‌کنندهٔ دیگری** که مسئولیت را بر عهده بگیرد؟»

صاحب‌خانه‌ای را تصور کنید که آشپزخانه‌اش را بازسازی می‌کند. دو انتخاب دارد: ابزار و دفترچهٔ راهنما بخرد و خودش بسازد، یا پروژه را به متخصصی بسپارد که کارش ساختن آشپزخانه است. **ابزارها سازهای منفعل‌اند؛ متخصص شریکی همکار است.** وقتی متخصص را استخدام می‌کنید، فقط یک نقشه دستش نمی‌دهید و نمی‌روید؛ او به حالت‌های مرزی یا اشتباهی در طرح اولیهٔ شما برمی‌خورد که او را وامی‌دارد مکث کند، دربارهٔ موازنه‌ها با شما مشورت کند و بعد ادامه دهد.

دامنهٔ کران‌دار در برابر بی‌کران

ابزار یا `API` استاندارد بر سازوکار «بفرست و فراموش کن» کار می‌کند: یک درخواست کاملاً قالب‌بندی‌شده انتظار دارد و یک پاسخ برمی‌گرداند. اما عامل در فضای حل‌مسئلهٔ **بی‌کران** `(Unbounded)` کار می‌کند. محیط‌های نرم‌افزاری واقعی پر از ساختارهای دادهٔ مبهم، نیازمندی‌های گمراه‌کننده و ترجیح‌های متناقض‌اند — معادل دیجیتالِ دیوارهای کج. به‌ندرت می‌توان همهٔ جزئیات لازم یک گردش‌کار پیچیده را بدون شفاف‌سازی چندنوبتی مشخص کرد.

> **مسئلهٔ `GOTO` در معماری عامل‌محور**
>
> چون دامنهٔ عامل بی‌کران است، چپاندن آن در پوشش یک ابزار استاندارد، معادل معماریِ دستور `GOTO` را وارد هماهنگ‌کنندهٔ شما می‌کند. وقتی عاملی همکار را صدا می‌زنید، جریان کنترل از زمینهٔ ساختارمند و مورد انتظار خارج می‌شود: عامل ممکن است به حالت وقفه برود، اطلاعات بیشتری بخواهد، و اگر کاربر نظرش عوض شود یا رهایش کند، شاید هرگز خروجی مورد انتظار را به فراخوان‌کننده برنگرداند.
>
> **راه‌حل:** با ایزوله‌کردن این مسیریابی همکارانه در لایهٔ `A2A`، لایهٔ ابزار `(MCP)` تمیز، قابل‌پیش‌بینی و کاملاً ساختارمند باقی می‌ماند.

کارت عامل و فهرست‌های ثبت

برای آنکه کشف ممکن شود، هر عامل با یک **کارت عامل** `(Agent Card)` تعریف می‌شود: «رزومهٔ» استانداردشدهٔ دنیای هوش مصنوعی که هویتی ماشین‌خوان فراهم می‌کند و سه چیز را مشخص می‌کند: **توانایی‌ها** (چه کارهایی می‌تواند انجام دهد)، **امنیت و انطباق** (سیاست‌های مدیریت داده و مجوزهای لازم) و **اسکیماهای تعامل** (دیگران چگونه باید با آن ارتباط بگیرند).

پس از تجهیز به کارت، عامل می‌تواند در یک **فهرست ثبت عامل** ثبت شود که هدفش تبدیل عامل‌ها به سرویس‌های کشف‌شدنی است:

1. **فهرست‌های عمومی (مارکت‌پلیس‌ها)** `(marketplace)`**:** مانند یک آژانس استعدادیابی جهانی، به توسعه‌دهندگان اجازه می‌دهند عامل‌های متخصصشان را برای کشف‌شدن به جهان عرضه کنند. این راه را باز

می‌کند که کسی برای یک صنعت خاص عامل متخصص بسازد و تخصصش را به هزاران هماهنگ‌کنندهٔ دیگر واگذار کند.

2. **فهرست‌های خصوصی:** درون سازمان، محیطی امن و حکمرانی‌شده فراهم می‌کنند. توسعه‌دهنده‌ای در شرکتی بزرگ می‌تواند عاملی بسازد که گردش‌کاری داخلی را خودکار می‌کند و آن را برای استفادهٔ بخش‌های دیگر ثبت کند، بی‌آنکه امنیت به خطر بیفتد.

پیاده‌سازی: دو حرکت توسعه

سمت عرضه — عرضهٔ عامل به‌عنوان عامل A2A:

- **تعریف کارت عامل:** مشخصات رسمی عامل.
- **پیاده‌سازی مجری عامل (لایهٔ ترجمه):** درخواست‌ها و پاسخ‌های A2A را به فراخوان‌های مشخصِ چارچوب عامل‌محور زیرین ترجمه می‌کند.
- **برپاکردن نقطهٔ پایانی A2A:** مجری باید به‌صورت نقطهٔ پایانی سازگار با پروتکل منتشر شود.

سمت تقاضا — اتصال به عامل‌های راه دور: برنامه به‌عنوان هماهنگ‌کننده عمل می‌کند؛ هسته‌ای مرکزی که بار شناختی‌اش صرف فهم قصد کاربر، مدیریت گردش‌کار کلان و واگذاری (handoff) کارهای مشخص به عامل‌های تخصصی راه دور می‌شود. اتصال معمولاً یکی از دو الگو را دنبال می‌کند:

```
# 1. Direct instantiation via a hardcoded endpoint
# Ideal for specific vendor integrations or private agents
billing_specialist = RemoteA2aAgent(
    name="billing_agent",
    endpoint="https://api.vendor.com/v1/billing/a2a",
)

# 2. Indirect instantiation via an Agent Registry
# The registry resolves resource names and handles authentication validation
registry = AgentRegistry(project_id=project_id, location=location)
agent_name = f"projects/{project_id}/locations/{location}/agents/YOUR_AGENT_ID"
my_remote_agent = registry.get_remote_a2a_agent(agent_name=agent_name)
```

لایهٔ توسعه‌پذیری و درآمدزایی

با حل پراکندگی، A2A لایهٔ ارتباطی یکپارچه‌ای می‌سازد که می‌توان روی آن ساخت. هستهٔ پروتکل ستون فقرات انتقال و مذاکره است، اما برنامه‌های تراکنشی غنی اغلب توانایی‌های بسیار تخصصی می‌خواهند. این نیاز از راه سازوکار

افزونه‌های A2A پاسخ می‌گیرد: الگویی استاندارد که به عامل‌ها اجازه می‌دهد قابلیت‌های اختیاری و مرتبه‌بالاتر را امن اعلام، مذاکره و اجرا کنند. سه چارچوب بنیادی که به‌شکل افزونهٔ بومی روی این پایه کار می‌کنند عبارت‌اند از A2UI، UCP و AP2.

کارکردن عامل تنها نیمی از نبرد است؛ نیمهٔ دیگر پایداری تجاری بلندمدت است. به‌دنبال موفقیت الگوی «نرم‌افزار به‌مثابه سرویس»، A2A به‌طور طبیعی مدل **«عامل به‌مثابه سرویس»** را ممکن می‌کند: عرضهٔ عامل‌های تخصصی در مدل مصرف‌محور از راه کانال‌های فروش گوناگون، از مارکت‌پلیس‌های ابری با پشتیبانی بومی از قیمت‌گذاری ترکیبی (کارمزد ثابت به‌علاوهٔ مصرف) تا ریزتراکنش‌های بدون‌مجوزِ ماشین‌به‌ماشین. در الگوی اخیر، سرور درخواست پرداخت‌نشده را با کد وضعیت HTTP 402 و صورت‌حسابی ماشین‌خوان برمی‌گرداند؛ عامل فراخوان، پرداخت را خودگردان انجام می‌دهد و درخواست را با توکن اثبات پرداخت رمزنگاشتی تکرار می‌کند.

۷. A2UI: رابط کاربری به‌عنوان خروجی عامل

شکاف ارتباطی

وقتی از همکارتان می‌پرسید «عملکرد فصل چهارم به تفکیک منطقه چطور بود؟»، او صفحه‌گسترده دستتان نمی‌دهد. نموداری میله‌ای روی تخته می‌کشد، مناطق برجسته را دور می‌گیرد و زمینه اضافه می‌کند. انسان‌ها بینش را این‌طور به اشتراک می‌گذارند: با تصویر و تعامل. اما عامل‌های ما JSON خام برمی‌گردانند و بعد خودتان نمودار را می‌سازید: وارد کردن کتابخانه، پیکربندی محورها، مدیریت وضعیت. این جابه‌جایی مدام میان «پرسیدن برای بینش» و «کار رابط کاربری»، جریان کدنویسی شهودی را می‌شکند.

رابط کاربری مولد

رابط کاربری مولد (Generative UI) یعنی مدل‌های زبانی در رانتایم و بر پایهٔ قصد و زمینهٔ کاربر، رابط بسازند. به‌جای آنکه توسعه‌دهنده هر حالت ممکن رابط را سخت‌کد کند، مدل رابط مناسب را به‌اقتضا تولید می‌کند. وقتی می‌پرسید «فروش فصل چهارم را به تفکیک منطقه مقایسه کن»، سامانه فقط داده بازیابی نمی‌کند: چیدمانی تعاملی، کارت‌ها، صافی‌ها و کنترل‌های متناسب با همان خواسته را می‌سازد.

> **چالش: انجام امنِ این کار**
>
> اجرای کد دلخواهِ رابط کاربری که یک مدل زبانی تولید کرده، ریسک‌های امنیتی جدی دارد: تزریق کد، حملهٔ XSS و عوارض جانبی کنترل‌نشده.

راه‌حل: اعلان قصد، نه تولید کد

A2UI استانداردی مستقل از چارچوب برای **اعلان قصد رابط کاربری** است: راهی متن‌باز که عامل‌ها رابط‌ها را در قالبی قابل‌حمل و اعلانی توصیف کنند، به‌جای جاری‌کردن دادهٔ خام یا فرستادن کد دلخواه.

> آهنگ‌ساز به نوازنده‌ها فایل ضبط‌شده نمی‌دهد؛ نُت می‌دهد. همان نُت روی پیانو، ارکستر یا سینتی‌سایزر نواخته می‌شود و هر ساز نشانه‌گذاری را با صدای خودش تفسیر می‌کند.

A2UI نُت موسیقیِ رابط کاربری است: عامل قصد را می‌نویسد (چه چیزی رندر شود و چطور ترکیب شود) و هر رندرگری آن را به‌صورت بومی روی دستگاهی که کاربر واقعاً دارد اجرا می‌کند. عامل لازم نیست بداند مقصد، وب است یا موبایل یا پوشیدنی؛ فقط کاتالوگ (Catalog) مؤلفه‌های موجود را می‌داند.

چرا این جداسازی، A2UI را امن می‌کند

عامل کد اجرایی تولید نمی‌کند (که کابوس امنیتی است) و پیکسل از پیش رندرشده هم نمی‌فرستد (که نمی‌تواند بازچینش شود یا تعاملی بماند). به‌جایش از **کاتالوگی مورد اعتماد** درخواست مؤلفه می‌کند و کلاینت با کتابخانهٔ مؤلفهٔ خودش رندر می‌کند. کاتالوگ تعیین می‌کند چه چیزی در دسترس است، عامل تصمیم می‌گیرد چطور چیده شود، و کلاینت ساختار نهایی را سرِهم می‌کند.

کاتالوگ پایه و ساخت کاتالوگ خودتان

A2UI یک کاتالوگ پایه با ۱۸ مؤلفهٔ آمادهٔ استفاده برای نمونه‌سازی و نمایش عرضه می‌کند:

دسته	مؤلفه‌ها
چیدمان	`Row, Column, List`
نمایش	`Text, Image, Icon, Divider`
ظرف	`Card, Modal, Tabs`
رسانه	`Video, AudioPlayer`
تعاملی	`Button, TextField, CheckBox, Slider, DateTimeInput, ChoicePicker`

این مجموعه در نسخهٔ `v0.8` «استاندارد» نامیده می‌شد و در `v0.9` به «پایه» تغییر نام یافت؛ نشانه‌ای عمدی که رابط‌های تولیدی باید **کاتالوگ خودشان** را بیاورند و مؤلفه‌های موجودشان (دکمه‌های سامانهٔ طراحی، نمودارها، نقشه‌ها) را به انواع `A2UI` نگاشت کنند. عامل تغییر نمی‌کند؛ فقط نگاشت کاتالوگِ رندرگر عوض می‌شود.

نمونهٔ یک پیام `A2UI` (قالب نسخهٔ `v0.9`):

```
{
  "version": "v0.9",
  "updateComponents": {
    "surfaceId": "main",
    "components": [
      {"id": "root", "component": "Column",
       "children": ["title", "summary", "export"]},
      {"id": "title", "component": "Text",
       "text": "Q4 Sales", "variant": "h1"},
      {"id": "summary", "component": "Text",
       "text": "Revenue grew 12% QoQ"},
      {"id": "export", "component": "Button", "child": "export-label",
       "action": {"event": {"name": "export_csv"}}},
      {"id": "export-label", "component": "Text", "text": "Export CSV"}
    ]
  }
}
```

مؤلفه‌ها فهرستی مسطح از مجاورت‌ها را می‌سازند که با شناسه به هم ارجاع می‌دهند. همین ساختار، هم تولید تدریجی را برای مدل زبانی آسان می‌کند و هم به‌روزرسانی را برای کلاینت، بدون رندر دوبارهٔ کل صفحه.

دو الگوی تولید `A2UI`

در عمل دو راه برای تولید وجود دارد و انتخاب، بیشتر به این بستگی دارد که **تصمیم چیدمان کجا گرفته می‌شود:**

- **پیش‌فرض — مدل مستقیماً `A2UI` تولید می‌کند:** مدل مالک چیدمان است و آن را با قصد کاربر تطبیق می‌دهد؛ همان عامل برای «این منطقه‌ها را مقایسه کن» و «روندها را نشان بده» دو رابط متفاوت می‌سازد.
- **تخصصی — ابزار ساختار ثابتی برمی‌گرداند:** یک فراخوان ابزار، بدون صرف توکن برای ساخت رابط، با خروجی کاملاً قابل‌پیش‌بینی. انتخاب درست وقتی است که چیدمان از روی ورودی‌ها قطعی `(deterministic)` است: داشبورد فروش هر منطقه یک شکل دارد و هر فرم رزرو همان فیلدها را. ابزار در عمل یک قالب سمت سرور است.

پرسش	خروجی مناسب
«میانگین چقدر است؟»	داده به‌صورت متن
«این منطقه‌ها را مقایسه کن»	رابط تولیدشده به‌دست مدل زبانی
«داشبوردم را نشان بده»	رابط ساخته‌شده به‌دست ابزار (قالب ثابت)
تماس برنامه به برنامه	دادهٔ JSON

بوم تعاملی

رابط‌های گفت‌وگومحور سنتی خطی‌اند و هر پاسخ ایستاست. رویکرد بوم متفاوت است: فضای کاری ماندگاری می‌سازد که هم عامل و هم کاربر می‌توانند ویرایشش کنند. به‌جای پیمایش تاریخچهٔ گفت‌وگو، سندی زنده دارید؛ عامل بخش‌ها را تغییر می‌دهد، شما دستی ویرایش می‌کنید و هر دو تغییر در زمان واقعی بازتاب می‌یابد. **رابط کاربری فقط رندر نمی‌شود؛ خودش رسانهٔ ارتباطی است.**

بهترین روش‌ها

- **دست‌نویسی JSON را کنار بگذارید.** از کیت توسعهٔ رسمی استفاده کنید: مدیر اسکیما (schema) سیستم پرامپتی می‌سازد که کاتالوگ و مثال‌های کارشده در آن تعبیه شده، کاتالوگ اعتبارسنج خودش را دارد و همان کیت، تجزیه‌گر بلوک‌های خروجی عامل را هم فراهم می‌کند.
- **اعتبارسنجی کنید و دوباره تلاش کنید.** در تولید، فراخوان ساخت رابط را در ساختار مدیریت خطا بپیچید و در صورت ناموفق‌بودن اعتبارسنجی اسکیما، به پاسخ متنی عقب‌نشینی کنید. خروجی مدل احتمالاتی است و رندرگر هرگز نباید بار بدشکل ببیند.
- **خروجی دوگانه بدهید.** هم داده و هم رابط را برگردانید تا مصرف‌کننده انتخاب کند: کلاینت‌های برنامه‌ای فیلد ui را نادیده می‌گیرند و کلاینت‌های انسانی آن را رندر می‌کنند.

۸. تجارت عامل‌محور: UCP و AP2

بخش‌های پیشین بر تعامل عامل‌ها با ابزارها، عامل‌های دیگر و رابط‌های کاربری تمرکز داشتند و این تعامل‌ها عمدتاً عملیات «خواندن» بودند. با تکامل این سامانه‌ها، عامل‌ها باید به اجرای «کنش‌هایی» گذر کنند که پیامد مالی واقعی دارند.

سناریوی راهنما

ساعت دو بامداد است و شما و هم‌خانه‌هایتان گرسنه‌اید، اما آن‌قدر مشغول درس‌خواندن هستید که نمی‌توانید متوقف شوید. پس دستیار هوشمندتان را می‌فرستید غذا بگیرد. ببینیم UCP و AP2 چطور کار را بین خودشان تقسیم می‌کنند.

UCP — پروتکل جهانی تجارت

پیش‌تر، هوش مصنوعی باید مرورگر را باز می‌کرد، به وب‌سایت بدطراحی رستوران می‌رفت، می‌کوشید تیک «گواکامولهٔ اضافه» را بزند و امیدوار بود سایت از کار نیفتد. UCP مثل یک مترجم جهانی عمل می‌کند: هر رستوران منو، ساعت کاری و گزینه‌های سفارشی‌سازی را به زبان ماشینی تمیز و استانداردی منتشر می‌کند.

عامل شما با UCP از سامانهٔ رستوران می‌پرسد «هنوز بازید؟ بوریتوی گیاهی دارید؟» و برای ساختن سفارش می‌گوید «یک بوریتوی مرغ، بدون پیاز، با خامهٔ ترش، به‌علاوهٔ یک چیپس»؛ و رستوران با مالیات، هزینهٔ ارسال و زمان تخمینی تحویل پاسخ می‌دهد. **پیش از UCP، برنامه می‌توانست کشف و اطلاعات‌گیری کند، اما تعامل با هر تأمین‌کننده باید جداگانه هماهنگ می‌شد.**

AP2 — پروتکل پرداخت عامل‌ها

حالا غذا در سبد است و عامل باید پرداخت کند. بدیهی است که شمارهٔ کارت واقعی‌تان را در پرامپت تایپ نمی‌کنید و نمی‌گویید «هر کاری خواستی بکن». AP2 پروتکل باز و مشترکی است که زبان مشترکی برای تراکنش‌های امن و منطبق میان عامل‌ها و فروشندگان فراهم می‌کند و اجازه می‌دهد عامل فقط **درون قواعدی که شما تعیین کرده‌اید** خرج کند:

- **گاردریل (guardrail) (مجوزنامه (Mandate)):** پیش از فرستادن عامل، یک قاعدهٔ دیجیتال را تأیید می‌کنید: «تا سقف ۲۵ دلار در این فروشگاه.»
- **دست‌دادن:** هنگام تسویه، شمارهٔ کارت نشان داده نمی‌شود؛ یک «سفتهٔ» دیجیتالِ رمزنگاشته و امضاشده ارائه می‌شود که می‌گوید «انسانِ من این سفارش ۱۸٫۵ دلاری را تأیید کرده است». بانک فروشنده فوری این امضای دیجیتال را راستی‌آزمایی می‌کند.
- **بدون هزینهٔ پنهان:** اگر رستوران بکوشد به‌جای ۱۸٫۵ دلار، پنجاه دلار بگیرد، پروتکل فوری جلویش را می‌گیرد، چون قاعده‌ای را که شما امضا کرده‌اید نقض می‌کند.

	UCP	AP2
نقش	مغزی که تصمیم می‌گیرد چه بخرد، منو را می‌فهمد و غذا را در سبد می‌گذارد.	کیف پولی که امن می‌پردازد، بی‌آنکه سرتان کلاه برود.
یکپارچگی با	هر تأمین‌کنندهٔ کسب‌وکار	سامانه‌های پرداخت
ارزش‌های کلیدی	یکپارچگی واحد، زبان مشترک، معماری توسعه‌پذیر، رویکرد امنیت‌محور	مجوزدهی و ممیزی‌پذیری، اصالت قصد، مهار خطا و توهم عامل، پاسخ‌گویی

۹. جمع‌بندی

با پذیرش استانداردهای بنیادینی مانند MCP، A2A، A2UI، AP2 و UCP، سازمان‌ها می‌توانند بدهی فنی کمرشکنِ یکپارچه‌سازی‌های اختصاصی را حذف کنند و تمام تمرکزشان را روی هماهنگ‌کردن منطق باارزش کسب‌وکار بگذارند. این تغییر پارادایم، توسعه‌دهنده را از مکانیکی که APIهای شکننده را سیم‌کشی می‌کند، به معمار واقعی یک نیروی کار خودگردان جهانی ارتقا می‌دهد. با بلوغ این لایه‌های ارتباطی استانداردشده، صرفه‌های مقیاس کاملاً تازه‌ای گشوده می‌شود و شیوهٔ ساخت، مصرف و درآمدزایی نرم‌افزار سازمانی دگرگون می‌شود.

۱۰. خودآزمایی

1. در تیم شما چند مدل و چند ابزار در کارند؟ عدد N×M را حساب کنید. چند نقطهٔ یکپارچه‌سازی سفارشی همین حالا دارید؟
2. آخرین سروری که به عاملتان وصل کردید از کدام یک از سه منبع کشف بود؟ آیا کدش را خواندید؟
3. برای یک کار محلی و یک کار مستقرشده، کدام حامل را انتخاب می‌کنید و چرا؟
4. یک فراخوان ابزارِ ناموفق را به یاد بیاورید. اگر آن روز بازرس داشتید، اول کدام سه چیز را نگاه می‌کردید؟
5. کدام یک از شش «نباید» در پروژهٔ فعلی شما همین حالا نقض می‌شود؟
6. سه نشانه بگویید که عامل یکپارچهٔ شما به سقف معماری خورده است.
7. در پروژهٔ خود، عامل را به کدام سه زیرعامل تخصصی می‌شکنید؟ هر کدام چه ابزارهایی باید داشته باشند و چه ابزارهایی نباید؟
8. برای کدام‌یک از این‌ها MCP مناسب است و برای کدام A2A: تبدیل ارز، بازبینی قرارداد حقوقی، رزرو سفر چندمرحله‌ای؟ معیار تصمیم شما چیست؟
9. چرا A2UI امن‌تر از این است که عامل مستقیم کد رابط تولید کند؟ در یک جمله پاسخ دهید.
10. اگر قرار بود برای عامل خودتان یک «مجوزنامه» بنویسید، چه قیدهایی در آن می‌گذاشتید؟

منبع: این جزوه خلاصه و بازنویسی آموزشی مقالهٔ `Agent Tools & Interoperability` نوشتهٔ کانچانا پاتلولا، لوکاش اولینیچاک و پیِر پائولو ایپولیتو است (مجموعهٔ مقالات عامل‌ها، می ۲۰۲۶). ترجمهٔ کامل فارسی در همین پوشه در دسترس است.

واژه‌نامهٔ این جلسه

اصطلاحاتی که در این سند به کار رفته‌اند، به‌همراه معادل انگلیسی‌شان. فهرست کامل 159 اصطلاح دوره در واژه‌نامهٔ دوره آمده است.

اصطلاح	معادل انگلیسی	توضیح
ابزار	`tool`	تابعی که عامل می‌تواند فرا بخواند تا در جهان بیرون کنش کند.
اسکیما	`schema`	ساختار تعریف‌شدهٔ داده یا ورودی ابزار.
اعتبارنامه	`credential`	کلید یا توکنی که هویت را اثبات می‌کند.
بازیابی	`retrieval`	آوردن دانش بیرونی به زمینه در لحظهٔ نیاز.
تأخیر	`latency`	زمان سپری‌شده تا دریافت پاسخ.
تجزیه	`decomposition`	شکستن کاری بزرگ به زیرکارهای اتمی.
تخصص‌گرایی داخلی	`Internal specialization`	افراز منطقی یک عامل یکپارچه به زیرعامل‌های هدفمند، بدون عبور از مرز شبکه.
تست	`test`	وارسی قطعی رفتار کد. «یونیت تست» = آزمون واحد.
توهم	`hallucination`	تولید بااطمینانِ محتوای نادرست به‌دست مدل.
توکن	`token`	واحد کوچکی از متن که مدل پردازش می‌کند.
حامل	`Transport`	مسیر انتقال پیام‌های JSON-RPC: stdio برای محلی و SSE روی HTTP برای راه دور.
حلقه	`loop`	چرخهٔ ادراک ← برنامه ← کنش ← مشاهده که عامل در آن می‌چرخد.
خودگردان	`autonomous`	توان اجرای کار بدون تأیید انسانی در هر گام.
دامنهٔ کران‌دار / بی‌کران	`Bounded`	ابزار در دامنهٔ کران‌دار «بفرست و فراموش کن» کار می‌کند؛ عامل در دامنهٔ بی‌کران به گفت‌وگوی چندنوبتی نیاز دارد.

اصطلاح	معادل انگلیسی	توضیح
دروازه	`gate`	نقطهٔ وارسی که اجرا را تا برآورده‌شدن شرطی متوقف می‌کند.
دستور	`instruction`	قاعده‌ای که رفتار عامل را شکل می‌دهد، در برابر داده‌ای که پردازش می‌کند.
دیباگ	`debug`	یافتن و رفع علت ریشه‌ای خطا.
رابط کاربری مولد	`Generative UI`	ساخت پویای رابط کاربری در رانتایم بر پایهٔ قصد کاربر، به‌جای کدنویسی همهٔ حالت‌های ممکن.
رانتایم	`runtime`	محیطی که عامل یا برنامه در آن اجرا می‌شود.
رقیق‌شدن توجه	`Attention dilution`	هرچه ابزارهای یک عامل بیشتر شود، فضای جست‌وجوی کنش بعدی بزرگ‌تر و تصمیم‌گیری بدتر می‌شود.
رهبر ارکستر	`Conductor`	کار هم‌زمان و دستی با عامل در ویرایشگر؛ کنترل دقیق بر هر تغییر.
زمینه	`context`	اطلاعاتی که عامل در لحظه در اختیار دارد. پایهٔ «مهندسی زمینه» و «پوسیدگی زمینه».
زمینهٔ ایستا / پویا	`Static`	ایستا همیشه بارگذاری می‌شود (هویت عامل)؛ پویا فقط هنگام نیاز فراخوانی می‌شود (مهارت، نتیجهٔ ابزار، بازیابی سند).
زیرعامل	`subagent`	عاملی که عامل دیگری برای کاری مشخص فرا می‌خواند.
سندباکس	`sandbox`	محیط اجرای ایزوله و کم‌امتیاز برای مهار شعاع انفجاری.
سیاست	`policy`	قاعدهٔ حکمرانی که بیرون از مدل تعریف و اعمال می‌شود.

اصطلاح	معادل انگلیسی	توضیح
عامل	`agent`	مدل به‌علاوهٔ مهار؛ توان استدلال، به‌کارگیری ابزار و اجرای کار.
عامل یکپارچه	`Single-agent monolith`	یک عامل با ده‌ها ابزار و یک سیستم پرامپت غول‌آسا؛ سریع ساخته می‌شود، اما به سقف مقیاس می‌خورد.
فراخوان ابزار	`tool call`	یک نوبت مشخص از به‌کارگیری ابزار، با آرگومان‌ها و نتیجه‌اش.
فهرست ثبت	`Registry`	جایی که عامل‌ها به سرویس‌های کشف‌شدنی بدل می‌شوند؛ عمومی (مارکت‌پلیس) یا خصوصی (سازمانی).
قصد / نیت	`intent`	خواستهٔ واقعی کاربر، در برابر آنچه لفظاً گفته. ریشهٔ «انحراف قصد» و «رضایت قصد».
قطعی / غیرقطعی	`deterministic`	قطعی یعنی ورودی یکسان همیشه خروجی یکسان می‌دهد؛ مدل‌ها غیرقطعی‌اند.
مارکت‌پلیس	`marketplace`	بازارگاه عرضهٔ عامل‌ها یا مهارت‌ها.
مجوزنامه	`Mandate`	قاعدهٔ امضاشدهٔ کاربر که سقف و شرایط خرج‌کردن عامل را تعیین می‌کند.
مرز	`boundary`	خط جدایی میان دو ناحیهٔ اعتماد یا مسئولیت.
مسیر	`trajectory`	توالی گام‌های استدلال و فراخوان ابزار که عامل تا رسیدن به پاسخ طی می‌کند.
مسیریابی	`routing`	انتخاب اینکه کدام مهارت یا عامل کار را بردارد.
مشخصات	`spec`	سند قصد و رفتار مورد انتظار؛ منبع حقیقت برای عامل و انسان.

اصطلاح	معادل انگلیسی	توضیح
مهار	`harness`	هرچه پیرامون مدل قرار می‌گیرد: پرامپت، ابزار، سندباکس، ارکستراسیون، قلاب. عامل = مدل + مهار.
مهارت	`skill`	پوشهٔ دانش رویه‌ای که عامل فقط هنگام نیاز بارگذاری می‌کند.
مهندسی عامل‌محور	`Agentic engineering`	همان قدرت مولد، اما درون سامانه‌ای از مشخصات، تست‌ها، ارزیابی‌ها و گاردریل‌ها با نظارت انسانی بر معماری.
هماهنگ‌کننده	`Orchestrator`	واگذاری ناهم‌زمان کار به چند عامل و بازبینی نتیجه؛ کار در سطح انتزاع بالاتر.
هویت	`identity`	اینکه چه کسی یا چه عاملی دارد کنش می‌کند.
وابستگی	`dependency`	کتابخانهٔ بیرونی که پروژه به آن تکیه دارد.
واگذاری	`handoff`	سپردن کار از عاملی به عامل دیگر.
پرامپت	`prompt`	متنی که به مدل داده می‌شود. پیش‌تر «درخواست» بود که با request اشتباه می‌شد.
پروتکل زمینهٔ مدل	`MCP`	سوکت استاندارد میان مدل و ابزارها؛ جایگزین پوشش‌های اختصاصی REST برای هر اتصال.
پوشش	`coverage`	سهمی از رفتار که آزمون یا ارزیابی واقعاً می‌سنجد.
کاتالوگ	`Catalog`	مجموعهٔ مؤلفه‌های مورد اعتمادی که عامل فقط می‌تواند از میان آن‌ها درخواست کند؛ ستون امنیت A2UI.
کارت عامل	`Agent Card`	«رزومهٔ» ماشین‌خوان عامل: توانایی‌ها، سیاست‌های امنیتی و اسکیمای تعامل.

اصطلاح	معادل انگلیسی	توضیح
کانتکست ویندو	`context window`	بیشینهٔ توکنی که مدل در یک نوبت می‌تواند ببیند.
کدنویسی شهودی	`Vibe coding`	توصیف خواسته به زبان طبیعی، پذیرفتن خروجی مدل و بازگرداندن پیام خطا به مدل تا وقتی کار کند؛ بدون راستی‌آزمایی نظام‌مند.
گاردریل	`guardrail`	قید بیرونی که کنش‌های مجاز عامل را محدود می‌کند.
گردش‌کار	`workflow`	توالی گام‌های یک کار تکرارشونده.
